

# RAG (RETRIEVAL AUGMENTED GENERATION)

Retrieval Augmented Generation (RAG) is a technique that enhances Large Language Models (LLMs) by providing them with relevant external knowledge. It has become one of the most widely used approaches for building LLM applications.

## Why use RAG?

RAG offers several advantages augmenting traditional methods of text generation, especially when dealing with factual information or data-driven responses. Here are some key reasons why using RAG can be beneficial:

- Access to fresh information
- Factual grounding
- Cannot access private or domain-specific data
- Can hallucinate

## RAG Architecture

**Step 1: Document Loader**

**Step 2: Text Splitter (Chunking)**

**Step 3: Embedding**

**Step 4: Vector DB**

**Step 5: User Query**

**Step 6: Retriever**

**Step 7: Create Prompt (User Query + retrieved relevant data)**

**Step 8: LLM Generate Response**

**Architecture Flow:-**

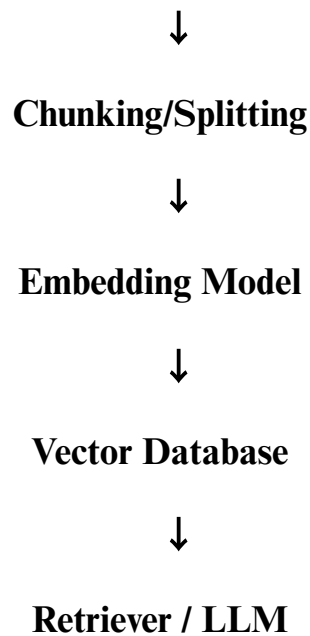
**Data Source**



**Document Loader**



**Text Extraction**



**Step 1: Document Loader:-** Document loaders provide a **standard interface** for reading data from different sources (such as Slack, Notion, or Google Drive) into LangChain's **Document** format. This ensures that data can be handled consistently regardless of the source.

### **Why Document Loaders Are Used**

Without loaders:

- Every file format needs custom parsing
- Different sources have inconsistent structures
- AI pipelines become complex

Document loaders solve this by:

- Standardizing data ingestion
- Simplifying preprocessing
- Supporting many formats
- Automating extraction

### **Types of Document Loaders:-**

Document loaders are classified based on:

**1. Webpages**

**2. PDFs**

- 3. Cloud provider
- 4. Social platforms
- 5. Message services

1. Webpage Loaders

Webpage loaders extract content from websites, blogs, online documentation, and HTML pages.

These loaders are commonly used for:

- Knowledge base ingestion
- Documentation crawling
- FAQ retrieval systems
- Web scraping for AI assistants

Common Features

- HTML parsing
- Metadata extraction
- Recursive crawling
- URL-based ingestion

Examples:

| Loader                | Purpose                      |
|-----------------------|------------------------------|
| WebBaseLoader         | Loads webpage content        |
| SitemapLoader         | Loads URLs from XML sitemaps |
| RecursiveURLLoader    | Crawls linked webpages       |
| UnstructuredURLLoader | Extracts clean webpage text  |

2. PDF Loaders

PDF loaders extract text, tables, and metadata from PDF documents.

They are widely used in:

- Enterprise document systems
- Research papers
- Legal contracts
- Medical reports
- Financial statements

- **Common Features**

- Text extraction
- OCR support
- Page-wise loading
- Table extraction
- Metadata parsing

- **Examples**

| Loader                | Purpose                 |
|-----------------------|-------------------------|
| PyPDFLoader           | Extracts text from PDFs |
| PDFPlumberLoader      | Better table extraction |
| UnstructuredPDFLoader | Handles complex layouts |
| OCR PDF Loader        | Reads scanned PDFs      |

### 3. Cloud Provider Loaders

Cloud loaders connect to cloud storage services and enterprise platforms. These loaders help organizations ingest large-scale internal data securely.

#### Supported Cloud Sources

- Google Drive
- AWS S3
- Azure Blob Storage
- SharePoint
- Dropbox

#### Common Features

- API-based authentication

- Automatic synchronization
- File filtering
- Incremental updates

## Examples

| Loader                 | Cloud Service        |
|------------------------|----------------------|
| S3FileLoader           | AWS S3               |
| GoogleDriveLoader      | Google Drive         |
| AzureBlobStorageLoader | Azure Blob Storage   |
| SharePointLoader       | Microsoft SharePoint |

## 4. Social Platform Loaders

Social platform loaders collect content from social media and community platforms.

These loaders are useful for:

- Sentiment analysis
- Trend monitoring
- Customer feedback systems
- Social intelligence applications

### Supported Platforms

- Twitter/X
- Reddit
- LinkedIn
- Facebook
- YouTube comments

### Common Features

- API integration
- Comment extraction
- Post retrieval
- User metadata collection

**Examples**

| Loader         | Platform                   |
|----------------|----------------------------|
| TwitterLoader  | Twitter/X posts            |
| RedditLoader   | Reddit discussions         |
| YouTubeLoader  | Video transcripts/comments |
| LinkedInLoader | Professional content       |

**5. Message Service Loaders**

Message service loaders ingest data from communication and collaboration platforms.

These are commonly used in:

- Enterprise chatbots
- Internal support systems
- Collaboration analytics
- Knowledge management systems

**Supported Services**

- Slack
- Microsoft Teams
- Discord
- Telegram
- WhatsApp exports

**Common Features**

- Channel/message extraction
- Conversation threading
- Timestamp metadata
- User mapping

**Examples**

| Loader         | Service               |
|----------------|-----------------------|
| SlackLoader    | Slack messages        |
| TeamsLoader    | Microsoft Teams chats |
| DiscordLoader  | Discord conversations |
| TelegramLoader | Telegram exports      |

---

## Step 2: Text Splitter (Chunking):-

Text splitters break large docs into smaller chunks that will be retrievable individually and fit within model context window limit.

### What is a Chunk?

A chunk is a smaller piece of a large document.

### Why Text Splitters Are Needed

Large Language Models (LLMs) cannot process infinitely large text.

Every model has a:

- **Context Window**
- **Token Limit**

Example:

- GPT models have token limits
- Embedding models also have limits

If a document is too large:

- It gets truncated
- Important information is lost
- Retrieval quality decreases

## 1. Character Text Splitter

Character text splitters divide text based on a fixed number of characters.

This is the simplest chunking strategy and is useful for:

- Small documents
- Basic preprocessing
- Fast chunk generation

### Features

- Fixed chunk size
- Optional chunk overlap
- Simple implementation

### Advantages

- Fast processing
- Easy to configure

### Limitations

- May break sentences or context
- Lower semantic quality

## 2. Recursive Character Text Splitter

Recursive splitters intelligently divide text using multiple separators such as:

- Paragraphs
- Sentences
- Spaces
- Characters

This is one of the most commonly used splitters in RAG systems.

### Features

- Preserves semantic structure
- Better context retention
- Flexible chunking

### Advantages

- Better retrieval accuracy
- Maintains readability

### Limitations



- Slightly slower than fixed splitting

### 3. Token Text Splitter

Token-based splitters divide text according to token count instead of character count.

This method is optimized for LLM token limits.

#### Features

- LLM-aware chunking
- Accurate token management
- Reduces context overflow

#### Common Use Cases

- OpenAI models
- Claude models
- Gemini models
- Production RAG systems

#### Advantages

- Efficient LLM usage
- Better context optimization

#### Limitations

- Requires tokenizer support

### 4. Sentence Splitter

Sentence splitters divide text sentence by sentence.

This approach ensures that chunks preserve complete thoughts.

#### Features

- Sentence boundary detection
- Natural language preservation
- Context-friendly chunks

#### Advantages

- Preserves readability
- Better semantic meaning

#### Limitations

- Uneven chunk sizes

## 5. Semantic Text Splitter

Semantic splitters divide text based on meaning rather than fixed sizes.

These splitters use embeddings or NLP techniques to identify topic boundaries.

### Features

- Meaning-aware chunking
- Topic preservation
- High retrieval relevance

### Advantages

- Best retrieval quality
- Improved contextual relevance

### Limitations

- Computationally expensive
  - Slower processing
- 

## Step 3: Embedding:-

Converting text to numbers

Why Embeddings are Used in RAG

- Convert text into vectors for machine understanding
- Enable semantic (meaning-based) search
- Retrieve relevant context efficiently
- Improve answer accuracy and relevance
- Handle synonyms and similar phrases
- Reduce hallucinations using retrieved knowledge
- Support fast search in large document datasets

Example of Embedding in RAG

**Text:**

- “How to reset my password?”
- “Steps to recover account password”

**Embedding Output (vector representation):**

- [0.12, -0.45, 0.78, ...]

- [0.10, -0.40, 0.81, ...]

## How it works

1. **Vectorization** — The model encodes each input string as a high-dimensional vector.
2. **Similarity scoring** — Vectors are compared using mathematical metrics to measure how closely related the underlying texts are.

## Similarity metrics

Several metrics are commonly used to compare embeddings:

- **Cosine similarity** — measures the angle between two vectors.
- **Euclidean distance** — measures the straight-line distance between points.
- **Dot product** — measures how much one vector projects onto another.

## Top Embedding Models in LangChain (RAG Use)

- **OpenAIEmbeddings** Most widely used, strong semantic understanding, best general-purpose choice for RAG.
  - **CohereEmbeddings** Good multilingual support and high-quality retrieval performance for enterprise use cases.
  - **GoogleGenerativeAIEmbeddings** Scalable and optimized for Google ecosystem applications.
  - **MistralAIEmbeddings** Efficient and strong performance for modern LLM-based RAG systems.
  - **VoyageAIEmbeddings** High accuracy embeddings focused on retrieval quality.
  - **NomicEmbeddings** Lightweight, fast, and open-source-friendly option.
  - **OllamaEmbeddings** Local embedding model option for offline / privacy-focused RAG systems.
  - **HuggingFaceEmbeddings (Sentence Transformers)** Popular open-source choice with models like BGE and MiniLM for cost-free deployment.
- 

## Step 4: Vector store :-

A vector stores embedded data and performs similarity search.

Features in vector store:-

- **Stores embeddings** Saves text data as high-dimensional vectors instead of raw text.
- **Enables similarity search** Finds documents whose vectors are closest to the query vector (semantic match).
- **Improves retrieval speed** Uses optimized indexing (e.g., HNSW, FAISS) for fast searching across large datasets.
- **Acts as knowledge memory** Keeps external knowledge that the LLM can query during generation.
- **Supports scalable RAG systems** Handles millions of documents efficiently.
- **Bridges query and context** Converts user query into vector and retrieves most relevant context for the LLM.



| Feature                         | Vector Store                                                                                | Vector Database                                                                                                     |
|---------------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Definition</b>               | A system that stores and retrieves vector embeddings, usually as a simple abstraction layer | A full-fledged database system designed specifically for storing, indexing, and querying vector embeddings at scale |
| <b>Complexity</b>               | Simple / lightweight                                                                        | Complex / enterprise-grade                                                                                          |
| <b>Scope</b>                    | Limited functionality (store + similarity search)                                           | Full database capabilities (storage, indexing, filtering, scaling, persistence, etc.)                               |
| <b>Indexing support</b>         | Basic or external indexing (sometimes none)                                                 | Advanced indexing methods like HNSW, IVF, PQ, etc.                                                                  |
| <b>Scalability</b>              | Limited scalability                                                                         | Highly scalable (millions to billions of vectors)                                                                   |
| <b>Persistence</b>              | Sometimes in-memory or file-based                                                           | Fully persistent storage with durability guarantees                                                                 |
| <b>Filtering support</b>        | Limited metadata filtering                                                                  | Advanced hybrid search (vector + metadata filtering)                                                                |
| <b>Query capabilities</b>       | Simple similarity search (k-NN)                                                             | Advanced queries (hybrid search, filtering, ranking, reranking)                                                     |
| <b>Performance optimization</b> | Basic                                                                                       | Highly optimized for low-latency + high throughput                                                                  |
| <b>Use cases</b>                | Small apps, prototypes, experiments                                                         | Production-grade AI systems, enterprise search, RAG at scale                                                        |
| <b>Examples</b>                 | FAISS (often used as store), Chroma (light mode)                                            | Pinecone, Weaviate, Milvus                                                                                          |

|                         |                                   |                                              |
|-------------------------|-----------------------------------|----------------------------------------------|
|                         |                                   |                                              |
| <b>Deployment style</b> | Often embedded in app             | Standalone service (cloud or self-hosted)    |
| <b>Data handling</b>    | Mostly raw vectors                | Vectors + metadata + indexing + APIs         |
| <b>Reliability</b>      | Lower (depends on implementation) | High (fault tolerance, replication, backups) |

## Types of Vector Stores (used in RAG)

- **In-memory vector stores** Store embeddings in RAM for very fast search, but data is not persistent. *Example:* FAISS (basic usage)
  - **Disk-based vector stores** Store vectors on disk with indexing for persistence and scalability. *Example:* FAISS (with disk index), Annoy
  - **Open-source vector stores** Self-hosted solutions with full control and customization. *Example:* Chroma, Milvus, Qdrant, Weaviate
  - **Managed vector databases (cloud-based)** Fully managed, scalable services with APIs and infrastructure handled. *Example:* Pinecone, Weaviate Cloud, Zilliz Cloud
  - **Hybrid vector stores** Support both vector search + keyword/metadata filtering (hybrid search). *Example:* Weaviate, Qdrant
  - **Embedded / local vector stores** Run locally inside applications for offline or lightweight use cases. *Example:* Chroma (local mode), FAISS
- 

## Step 5: Retriever:-

A **retriever** is a core component in a RAG system that is responsible for **finding the most relevant pieces of information** from a large knowledge base in response to a user query.

### What Does a Retriever Do?

A retriever performs three main tasks:

- **1. Converts query into a searchable format**
- Usually converts text into embeddings (vectors)
- **2. Searches knowledge base**
- Finds similar documents using vector similarity
- **3. Returns top-k relevant chunks**
- Passes best matches to the LLM

# Types of Retrievers :-

## 1. Based on Data Sources

## 2. Based on Search Strategy

### 1. Types of Retrievers Based on Data Sources

This classification depends on **where the knowledge comes from**.

#### 1.1 Wikipedia Retriever

A Wikipedia retriever is used when:

- You need **general world knowledge**
- You want **trusted, structured, and publicly available information**
- You are building systems like chatbots, QA systems, or assistants that require factual grounding

It is especially useful when:

- You don't have proprietary data
- You want a fallback knowledge base
- You need high coverage of general topics

#### 1.2 Vector Store Retriever

Used when:

- You want **semantic search over large unstructured data**
- Exact keyword matching is insufficient
- You are working with **RAG over enterprise data, PDFs, websites, or knowledge bases**

This is the most common retriever in modern RAG systems.

#### 1.3 Document Retriever (Custom / Local Data)

Used when:

- You have **private or domain-specific data**
- Examples:
  - Company PDFs
  - Internal policies
  - CRM records
  - SQL databases
- You want full control over retrieval pipeline

## 2. Types of Retrievers Based on Search Strategy

This classification focuses on **how retrieval is optimized**.

## 2.1 MMR (Maximal Marginal Relevance) Retriever

### 📌 Need

Standard similarity search often returns:

- Highly similar documents
- Redundant information
- Poor coverage of different aspects of a query

MMR solves this by ensuring:

“Relevant results, but not repetitive ones”

It is needed when:

- You want **diverse context**
- You are feeding limited tokens to LLM
- You want balanced coverage of multiple subtopics.

## 2.2 Multi-Query Retriever

### 📌 Need

Users often ask:

- Vague questions
- Under-specified queries
- Questions with multiple interpretations

A single query may miss relevant documents.

Multi-query retrievers improve:

- **Recall**
- Coverage of different interpretations

## 2.3 Contextual Compression Retriever

### 📌 Need

Vector retrieval often returns:

- Large chunks of text
- Irrelevant sections inside relevant documents
- Token inefficiency for LLMs

Contextual compression solves:

“Keep only the useful parts of retrieved documents”

---



## **Step 7: Create Prompt (User Query + Retrieved Relevant Data)**

### **What is it?**

In this step, the system combines the user's query with the retrieved relevant information to create a structured prompt for the Large Language Model (LLM).

### **Why is it Needed?**

LLMs generate better and more accurate responses when they receive proper context. Instead of answering only from pre-trained knowledge, the model uses retrieved external data to generate context-aware answers.

### **How it Works**

1. User submits a query.
2. Retriever fetches relevant document chunks.
3. The system combines:
  - User Query
  - Retrieved Data
  - Instructions/System Prompt
4. A final prompt is created and sent to the LLM.

### **Example**

#### **User Query**

What is Retrieval-Augmented Generation?

#### **Retrieved Data**

RAG combines retrieval systems with Large Language Models to improve response accuracy using external knowledge.

#### **Final Prompt**

Answer the following question using the provided context.

#### **Question:**

What is Retrieval-Augmented Generation?

#### **Context:**

RAG combines retrieval systems with Large Language Models to improve response accuracy using external knowledge.

## Types of Prompt Creation

### 1. Simple Prompting

Directly combines query and retrieved data.

### 2. Template-Based Prompting

Uses predefined prompt structures for consistency.

### 3. Instruction Prompting

Adds instructions such as:

- “Answer briefly”
- “Use bullet points”
- “Do not hallucinate”

### 4. Chain-of-Thought Prompting

Encourages step-by-step reasoning before generating the answer.

## Pros

- Improves answer accuracy
- Reduces hallucination
- Provides contextual understanding
- Enables domain-specific responses

## Cons

- Large prompts increase token cost
  - Poor prompt formatting affects output quality
  - Too much context may confuse the model
- 

## Step 8: LLM Generate Response

### What is it?

The Large Language Model (LLM) processes the final prompt and generates a human-like response.

### Why is it Needed?

The LLM transforms retrieved information into meaningful, natural language answers that users can easily understand.

### How it Works

1. Receives the final prompt.
2. Understands query and context.
3. Applies reasoning and language generation.
4. Produces the final response.

## **Types of Responses**

### **1. Generative Response**

Creates detailed explanations in natural language.

### **2. Extractive Response**

Returns exact information from retrieved documents.

### **3. Summarized Response**

Condenses long information into short summaries.

### **4. Conversational Response**

Provides interactive chatbot-style answers.

## **Popular LLMs**

- GPT
- LLaMA
- Gemini
- Claude
- Mistral

## **Pros**

- Human-like communication
- Strong reasoning capability
- Supports multiple languages
- Generates contextual answers

## **Cons**

- Can generate hallucinated responses
  - High computational cost
  - Depends heavily on prompt quality
  - Large models require significant resources
- 

## **References:-**

## Step 1: Document Loader

- [LangChain Document Loaders](#)

## Step 2: Text Splitter (Chunking)

- [LangChain Text Splitters](#)

## Step 3: Embedding

- [OpenAI Embeddings Guide](#)

## Step 4: Vector Database

- [FAISS Library](#)
- [Pinecone Vector Database](#)

## Step 5: User Query

- Speech and Language Processing

## Step 6: Retriever

- [LangChain Retrievers](#)

## Step 7: Create Prompt (User Query + Retrieved Data)

- [Prompt Engineering Guide](#)

## Step 8: LLM Generate Response

- [OpenAI Models Documentation](#)
  - [Hugging Face Transformers](#)
-